Lightspeed Data Compute for the Space Era

Thomas Sandholm*, Bernardo A. Huberman*, Klas Segeljakt*, and Paris Carbone*†
* *RISE - Research Institutes of Sweden*
thomas.sandholm@ri.se, bernardo.huberman@ri.se, klas.segeljakt@ri.se, paris.carbone@ri.se
† *KTH Royal Institute of Technology*
parisc@kth.se

***Abstract*—**
While thousands of satellites photograph Earth every day, most of that data never makes it to the ground because downlink bandwidth simply cannot keep up. Processing data in the Low Earth Orbit (LEO) zone offers promising capabilities to overcome this limitation. We propose *SpaceCoMP*, a MapReduce-inspired processing model for LEO satellite mesh networks. Ground stations submit queries over an area of interest; satellites collect sensor data, process it cooperatively at light-speed using inter-satellite laser links, and return only the results. Our compute model leverages space physics to accelerate compute on LEO megaconstellations. Our distance-aware routing protocol exploits orbital geometry. On top, our bipartite match scheduling strategy places map and a reduce tasks within orbital regions while minimizing aggregation costs. We have simulated constellations of 1,000–10,000 satellites showcasing 61–79% improvement in map placement efficiency over baselines, 18–28% over greedy allocation, and 67–72% reduction in aggregation cost. SpaceCoMP demonstrates that the orbital mesh is not merely useful as a communication relay, as seen today, but can provide the foundations for faster data processing above the skies.

***Index Terms*—Space Computing, MapReduce, Earth Observation**

I. Introduction

Low-earth orbit (LEO) satellite constellations are generating data at rates that far exceed their capacity to transmit it to the ground. A single high-resolution Earth observation satellite produces 1–2 TB of imagery per day [21], yet typical ground station contacts, lasting only 5–15 minutes per orbit, can transfer merely a fraction of this volume over bandwidth-constrained radio-frequency (RF) downlinks. With mega-constellations now reaching unprecedented scale, Starlink exceeded 7,500 operational satellites in 2024 with over 40,000 planned, and the EU's IRIS^2 program commits 10.6B to European sovereign connectivity [6], [12], the aggregate data generation potential reaches petabytes per day. Yet global ground station infrastructure remains the bottleneck: RF spectrum is shared and regulated, weather causes signal attenuation, and ground stations must be within line-of-sight during brief orbital passes. This *downlink wall* fundamentally limits the value that can be extracted from space-based sensing.

The emergence of optical inter-satellite links (ISL) offers a transformative opportunity. Unlike RF links that must traverse Earth's atmosphere, optical ISLs operate in vacuum at the speed of light, approximately 47% faster than signals in terrestrial fiber [7], with bandwidth exceeding 10 Gbps per link [15]. Table I summarizes the key differences. Modern LEO satellites equipped with laser terminals form mesh networks where any satellite can route data to any other through a sequence of hops, creating a distributed infrastructure with high-bandwidth internal connectivity but constrained external bandwidth to ground. From a computational lense the LEO architecture features thousands of interconnected compute nodes and data is naturally partitioned across collectors. The expense of performing external I/O essentially mirrors similar conditions that motivated MapReduce in terrestrial data centers two decades ago [5]. A key insight is that *processing data in orbit before transmission can dramatically reduce the potential of downlink saturation*, converting terabytes of raw sensor data into megabytes of actionable results.

TABLE I
RF Ground Links vs. Optical Inter-Satellite Links (ISLs)

Property	RF Ground Links	Optical ISLs
Medium	Atmosphere (lossy)	Vacuum
Propagation speed	≈200,000 km/s	299,792 km/s (c)
Bandwidth	Shared spectrum, <1 Gbps	10–100+ Gbps/link
Availability	Weather-dependent	Always-on
Ground infrastructure	Required	Not required
Long-haul latency	High (via ground hops)	Low (direct path)
Atmospheric loss	Yes (rain, clouds)	None

However, distributed computing in LEO presents novel challenges absent from terrestrial systems. Satellites orbit at 7.5 km/s, completing a full orbit in approximately 95 minutes. The network topology changes continuously: inter-plane link distances vary by up to 40% over each orbital period as planes converge near the poles and diverge at the equator. Near polar regions, satellites from adjacent planes pass each other at relative velocities exceeding 15 km/s, forcing temporary link disconnection. Task placement must therefore account not just for hop count but for actual transmission distance and time-varying link geometry. Furthermore, satellites remain in proximity to a ground station for only 5–15 minutes

before handover [18], requiring coordination of data collection, processing, and result delivery within narrow windows. Existing cloud software, designed for static datacenter topologies, cannot be directly deployed: the assumptions underlying conventional schedulers, routers, and consistency protocols break down when the infrastructure itself is in constant motion.

We propose Space Collect-MapReduce Processing (SpaceCoMP), a distributed processing model designed for LEO mega-constellation ISL networks. Consider hundreds of satellites capturing imagery of a wildfire. How do we extract intelligence from such a rich, distributed dataset without saturating the downlink? The data is already naturally partitioned across collector satellites; the challenge is ensuring that computation happens close to the data with minimal shuffling, both within the orbital network and ultimately when transmitting results to ground. SpaceCoMP extends the MapReduce paradigm with a *Collect* phase that explicitly models sensor data acquisition from an area of interest (AOI), followed by Map processing on nearby satellites and Reduce aggregation before downlink.

Our approach draws inspiration from MapReduce [5], the NASA core Flight System (cFS)[1], and Wireless Sensor Network protocols such as Contract-Net [22] and Directed Diffusion [8]. We make the following contributions:

1) **The SpaceCoMP processing model**: A three-phase (Collect-Map-Reduce) abstraction for data-intensive workloads in LEO constellations that lets developers focus on application logic while the infrastructure handles orbital dynamics and task placement (Section II).
2) **Distance-optimized routing**: A routing algorithm that exploits time-varying cross-plane link geometry to minimize total transmission distance by 8–21% (depending on orbital inclination) without increasing hop count (Section III).
3) **Bipartite matching scheduler**: A map task allocation strategy based on maximum-weight matching in bipartite graphs, improving placement efficiency by 61–79% over random allocation and 18–28% over greedy baselines (Section IV).
4) **Reduce placement strategy**: A center-of-AOI reducer placement that reduces aggregation cost by 67–72% compared to routing results directly to the line-of-sight ground station (Section IV).

We validate SpaceCoMP through simulations modeling Walker Delta constellations with realistic ISL parameters across constellation sizes from 1,000 to 10,000 satellites.

sectionFrom Cloud to Space Computing: a Primer

This section introduces the LEO environment, including a primer of satellite networking (Section I-A). We then position our work relative to terrestrial cloud computing I-B and sensor and edge network I-C literature.

A. The Low-Earth-Orbit Computing Environment

Satellite mega-constellations share similarities with datacenter networking: fat-tree topologies, rack-aware placement, millisecond latencies, and multi-Gbps bandwidths. In particular, LEO satellite networks share the goal of interconnecting compute nodes but operate under fundamentally different physical constraints and capabilities possible in the vacuum.

1) Orbital Altitude and Its Implications: Earth-orbiting satellites occupy three primary altitude bands. Geostationary Earth Orbit (GEO) satellites at 35,786 km remain fixed relative to ground locations but suffer approximately 120 ms one-way latency. Medium Earth Orbit (MEO), used by GPS, spans 8,000–20,000 km with orbital periods of 2–12 hours. LEO, at altitudes of 300–2,000 km, has emerged as the preferred domain for mega-constellations due to properties critical for distributed computing [19], [26]:

Low latency: At typical altitudes of 500–600 km, one-way propagation delay to ground is only 2–4 ms, enabling round-trip times of approximately 10 ms [9], comparable to inter-datacenter latencies.

Short orbital period: LEO satellites complete an orbit in 90–95 minutes, traveling at 7.5 km/s and circling the globe roughly 15 times per day.

Distributed coverage: Each satellite's ground footprint spans only $\sim$1,000 km diameter, necessitating constellations of hundreds to thousands of satellites while also enabling data-parallel data access by nature across nodes.

2) Constellation Geometry: LEO satellites are organized into structured formations that determine network topology and thus task placement options:

Orbital planes and shells. Satellites following the same circular path form an *orbital plane*, maintaining fixed spacing relative to one another. Multiple planes sharing the same altitude and inclination, distributed uniformly around Earth, constitute a *shell*. The Walker Delta pattern [25] arranges N planes separated by $360/N$ in longitude, producing a "doughnut-shaped" coverage volume with wraparound in both axes, naturally forming the 2D-Torus topology that can be leveraged for task placement.

Inclination. The angle between an orbital plane and Earth's equator determines coverage latitude. For instance, Starlink uses 53° inclination for populated regions; Earth observation missions often use near-polar orbits (e.g., 87° as in our simulations) for global coverage.

Phase offset. Satellites across planes are staggered to maximize coverage uniformity. This geometric regularity enables SpaceCoMP's bipartite matching formulation: when an area of interest maps to a rectangular region of satellites, tasks and processors form a natural grid structure.

3) Inter-Satellite Links: The emergence of optical inter-satellite links (ISLs) has transformed LEO constellations from isolated satellites into interconnected mesh networks [15]. Operating in vacuum via laser terminals, ISLs achieve:

- **Speed**: Light travels at $c = 299{,}792$ km/s in vacuum, approximately 47% faster than in optical fiber [7].
- **Bandwidth**: Modern terminals achieve 10–100+ Gbps per link.

[1] https://etd.gsfc.nasa.gov/capabilities/core-flight-system/

- **Availability**: Continuous operation independent of weather or ground station visibility.

The +Grid topology. LEO satellites typically carry four laser terminals connecting to neighbors ahead and behind in the same plane, plus neighbors in adjacent planes left and right. This forms a +Grid (2D-Torus) topology [17] where routing follows Manhattan distances, i.e., the number of hops between satellites equals the sum of horizontal (cross-plane) and vertical (within-plane) steps.

4) Dynamic Topology: Unlike datacenter networks, LEO mesh topology changes continuously but *predictably*:

Static intra-plane distances. Satellites within a plane maintain formation; inter-satellite distance is constant (Equation 1).

Cyclic inter-plane distances. Orbital planes converge near poles and diverge at the equator. Cross-plane link distances vary sinusoidally over each orbital period (Equation 2), exceeding 40% variation for high-inclination orbits.

Polar crossover events. Near poles, satellites from adjacent planes pass each other at relative velocities exceeding 15 km/s. Cross-plane links must be disabled during these "seam" crossings [18].

Ascending vs. descending trajectories. Half the satellites in a plane travel northward (ascending), half southward (descending). Their extreme relative velocity prevents stable ISL links between them. This reveals an implicit mutual exlusion constraint when schedulling compute: it is only possible to select *only* ascending or *only* descending satellites for any computation, but not a mix.

Critically, all dynamics are deterministic given orbital parameters. Satellite positions can be predicted in advance using propagation models such as SGP4 [23]. It is therefore crucial to exploit this orbital predictability in both routing and task placement when performing computing in space.

B. Cloud Computing Background

SpaceCoMP draws on three research threads in terrestrial cloud computing: distributed task scheduling, the MapReduce programming model, and wireless sensor networks.

1) Distributed Task Scheduling: The challenge of mapping tasks to processors has evolved through several eras. Early multiprocessor systems recognized that naive scheduling causes thrashing; co-scheduling techniques modeled allocation as bin-packing to avoid fragmentation [14]. As HPC clusters became oversubscribed, backfilling [10] improved utilization by allowing small jobs to advance when they would not delay larger ones.

Cloud computing (2010–) shifted the paradigm toward horizontal scaling across commodity hardware. Kubernetes [3] schedules containers based on resource requirements, affinity rules, and image locality. Apache Spark's delayed scheduling [27] trades fairness for data locality by allowing jobs to wait briefly for preferred nodes. More recently, adaptivity has been applied to task placement for ML workloads [13] e.g., through the use of reinforcement learning. This allows "learning" placement policies that minimize execution time in heterogeneous GPU/CPU environments.

A distinct challenge emerges: the "datacenter" itself is in motion. Static affinity rules are insufficient when node proximity changes continuously. Space computing schedulers should therefore capture time-varying placement costs that terrestrial schedulers otherwise treat as constant.

2) MapReduce and Data-Parallel Processing: In the 2010s, Google revealed MapReduce [5] a compute paradigm that simplified distributed programming. While providing an over-simplified interface: `map` and `reduce` functions, MapReduce democratized automatic parallelization, fault tolerance, and data-locality-aware scheduling. Its key optimization was moving computation to data rather than vice versa, which greatly reduced costly network shuffles.

Despite the similarities exhibited in LEO satellite networks, the original MapReduce model would fail in orbit. Unlike terrestrial MapReduce, where "local" versus "remote" is binary, placement costs need to be remodelled to account for continuous functions of hop counts and physical distances. In addition, programming semantics are also missing to express pre-compute operations during sensor data acquisitions that precede processing.

C. Wireless and Edge Computing

1) Wireless Sensor Networks: Earth observation via satellite constellations have been previously studied within wireless sensor networks (WSN). In this context, sattelites are edge nodes that collect and aggregate environmental data. Problems of task allocation, relevant to this work, have been studied in similar settings. For example, the Contract-Net protocol [22] introduced competitive bidding for task allocation; distributed sensors bid based on their cost to complete tasks, and a controller minimizes total cost. Van der Horst et al. [24] also applied similar market-based allocation to satellite networks, though not for the ISL mesh topologies now prevalent. Directed Diffusion [8] demonstrated that routing nodes can actively aggregate results from distributed sensors, reducing data volume before transmission to a sink, in an analogous fashion to a reduce phase. In this work we capitalize on these ideas. As shown in III, it is possible to have collectors bid implicitly through the cost matrix, whereas, mappers can process locally, and reducers aggregate before the final hop to ground.

2) LEO Edge Computing: Recent work has begun exploring computation in LEO. Krios [2] introduces "availability zones" that mask satellite mobility, enabling terrestrial orchestrators to schedule predictively. Celestial [16] provides a virtual testbed for LEO edge emulation. SkyMemory [20] demonstrates distributed caching for transformer inference across satellites.

These systems treat LEO as an extension of terrestrial infrastructure. Yet, we aim for a programming model that embraces orbital dynamics as a first-class scheduling concern, optimizing task placement for the unique cost structure of time-varying ISL distances.

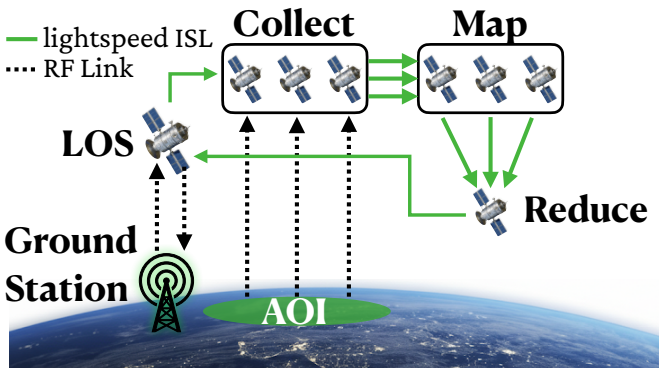

Fig. 1. SpaceCoMP Architecture

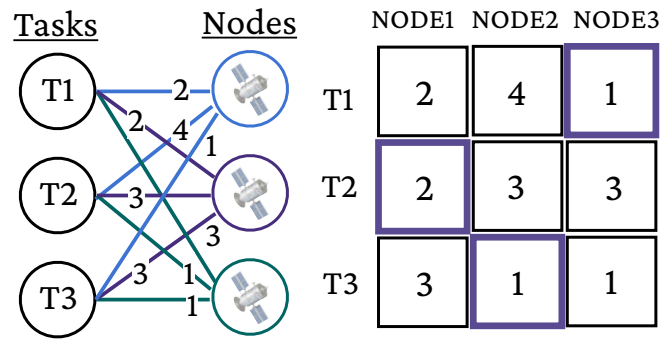

Fig. 2. Task Processor Cost Adjacency Matrix Example

II. System Overview

SpaceCoMP enables distributed data processing across LEO satellite constellations by extending the MapReduce paradigm with an explicit data collection phase. Figure 1 illustrates the architecture.

A. Request Flow

A ground station initiates processing by submitting a *job request* to the nearest visible satellite, termed the *Line-of-Sight (LOS) node* (Figure 1). The request specifies:

- An **Area of Interest (AOI)**: a geographic bounding box (latitude/longitude pairs) defining the region to observe.
- A **Collect task**: processes sensor configuration and data acquisition parameters.
- A **Map task**: processes local-compute logic applied to each collected data partition.
- A **Reduce task**: executes aggregation logic across all map outputs before a result downlink.

The LOS node acts as the *coordinator*, translating the geographic AOI into a grid of satellite nodes and orchestrating the three processing phases.

B. Processing Phases

Phase 1: Collect. The coordinator identifies satellites whose ground projections intersect the AOI. These *collector nodes* are instructed to acquire sensor data (e.g., capture imagery, record measurements). The data remains local to each collector, avoiding immediate network transfer.

Phase 2: Map. A subset of satellites within or near the AOI are designated as *mapper nodes*. The coordinator solves a bipartite matching problem (Section III-B) to assign each collector's data to a mapper, minimizing total transfer cost. Mappers execute the user-defined map function on their assigned data partitions, producing intermediate results.

Phase 3: Reduce. A single *reducer node*, selected to minimize aggregate transfer distance from all mappers (Section III-B3), collects intermediate results and applies the reduce function. The final output is routed to the LOS node for downlink to the ground station.

C. Design Rationale

SpaceCoMP's three-phase structure addresses the unique constraints of orbital networks:

- **Data locality**: By processing data near its collection point, SpaceCoMP minimizes ISL bandwidth consumption. Only reduced results traverse the long path to the LOS node.
- **Topology awareness**: Unlike terrestrial MapReduce, where data center topology is static, SpaceCoMP's scheduler accounts for time-varying inter-plane distances when computing placement costs.
- **Downlink optimization**: The reduce phase can compress petabytes of raw observations into megabytes of actionable intelligence before the bandwidth-constrained RF downlink.

The coordinator need not be the LOS node; any satellite with sufficient visibility could orchestrate the computation. However, placing coordination at the LOS node simplifies result delivery and ground station interaction.

III. System Model

We formalize the orbital network structure and task scheduling optimization that underpin SpaceCoMP.

A. Orbital Network Model

We assume a Walker Delta constellation shell comprising N orbital planes with M satellites per plane. Satellites are identified by tuples (s, o) where $s \in \{1, \ldots, M\}$ indexes position within a plane and $o \in \{1, \ldots, N\}$ indexes the orbital plane itself. All satellites share the same altitude h and inclination i, with planes uniformly distributed around the equator at $360/N$ separation. Within each plane, satellites are equidistant along the orbital path.

1) Network Topology: Each satellite maintains four inter-satellite links to its nearest neighbors: two within the same orbital plane (ahead and behind) and two in adjacent planes (left and right), forming a +Grid (2D-Torus) topology.

The **intra-plane distance** between adjacent satellites is constant:

$$D_m = (r_E + h)\sqrt{2\left[1 - \cos\frac{2\pi}{M}\right]} \tag{1}$$

where $r_E = 6{,}371$ km is Earth's radius. The **inter-plane distance** varies over the orbital period T as planes converge near the poles and diverge at the equator:

$$D_n(t) = (r_E + h)\sqrt{2\left[1 - \cos\left[\frac{2\pi}{N}\right]\right]} \times \sqrt{\cos^2\left[2\pi\frac{t}{T}\right] + \cos^2(i)\,\sin^2\left[2\pi\frac{t}{T}\right]} \tag{2}$$

where $t \in [0, T)$ is the time since the satellite crossed the equator (ascending), and the orbital period is:

$$T = 2\pi\sqrt{\frac{(r_E + h)^3}{\mu}} \tag{3}$$

with $\mu \approx 3.986 \times 10^{14}$ m^3/s^2 (Earth's gravitational parameter).

2) AOI→Satellite Mapping: An Area of Interest is specified as a geographic bounding box: upper-left and lower-right (latitude, longitude) pairs. We map satellite positions to ground coordinates using the Earth-Centered, Earth-Fixed (ECEF) system and the SGP4 propagation model [23].

A satellite may be in one of three states relative to an AOI at time t: *within AOI ascending*, *within AOI descending*, or *outside AOI*. To ensure stable ISL connectivity, SpaceCoMP selects only ascending *or* only descending satellites for any computation, never both, as their relative velocity (>15 km/s) prevents reliable links.

B. Task Scheduling Model

The core optimization goal is minimizing data movement: placing map tasks close to collectors, and the reducer close to mappers.

1) Problem Formulation: Given an AOI, let $\mathcal{T} = \{t_1, \ldots, t_k\}$ denote collector nodes (data sources) and $\mathcal{P} = \{p_1, \ldots, p_k\}$ denote candidate mapper nodes (processors). We seek an assignment $\sigma : \mathcal{T} \to \mathcal{P}$ minimizing total placement cost:

$$\min_{\sigma} \sum_{t \in \mathcal{T}} C(t, \sigma(t)) \tag{4}$$

subject to each mapper processing exactly one task (bijective assignment).

This is the *linear sum assignment problem*, solvable in $O(k^3)$ time via the Hungarian algorithm [**?**]. The problem can equivalently be viewed as finding a minimum-weight perfect matching in a bipartite graph where edge weights represent placement costs.

2) Cost Function: The cost of assigning task t to processor p comprises processing time and data transfer time:

$$C(t, p) = m_p \cdot K + h_{t,p} \cdot H + S(d_{t \to p}, V) \tag{5}$$

where:

- m_p is the map processing time factor
- K is a normalization constant (processing cost per unit)
- $h_{t,p}$ is the hop count from t to p
- H is the per-hop overhead (queuing, forwarding)
- $S(d, V)$ is the transmission time for V bytes over distance d

The transmission time accounts for propagation delay and channel capacity:

$$S(d, V) = \frac{d}{c} + \frac{V}{B\log_2(1 + \mathrm{SNR}(d))} \tag{6}$$

where c is the speed of light, B is channel bandwidth, and SNR decreases with distance due to free-space path loss:

$$\mathrm{SNR}(d) = \frac{P \cdot G_t \cdot G_r}{N \cdot \mathrm{FSPL}(d)}, \quad \mathrm{FSPL}(d) = \left(\frac{4\pi d}{\lambda}\right)^2 \tag{7}$$

with transmit power P, antenna gains G_t, G_r, wavelength λ, and noise power $N = k_B \cdot N_T \cdot B$ (Boltzmann constant k_B, noise temperature N_T).

The distance $d_{t \to p}$ is the sum of individual link distances along the Manhattan routing path, computed using Equations 1 and 2 for within-plane and cross-plane hops respectively.

3) Reduce Placement: After map tasks complete, their outputs must be aggregated at a reducer. We consider two placement strategies:

LOS placement: The reducer is the Line-of-Sight node coordinating the job. This minimizes the final hop to ground but may incur high aggregate transfer cost from distant mappers.

Center-of-AOI placement: The reducer is placed at the geometric center of the mapper distribution, minimizing total transfer distance from mappers. The reduced output then travels to the LOS node.

Overall, Center-of-AOI placement is advantageous when the reduce function significantly compresses data (high *reduction factor* F_R), as the long reducer→LOS transfer carries less volume.

IV. Evaluation

We evaluate SpaceCoMP through simulations modeling Walker Delta constellations with realistic ISL parameters. Our experiments address three questions:

1) Does distance-aware routing reduce transmission cost without increasing hop count?
2) Does bipartite matching outperform simpler task allocation strategies?
3) Does center-of-AOI reduce placement improve end-to-end cost?

A. Simulation Setup

Table II summarizes simulation parameters. We model constellations ranging from 1,000 to 10,000 satellites across 50–100 orbital planes at 87° inclination (typical for polar Earth observation missions). The AOI is defined as the satellite footprint over a large geographic region (United States) to ensure sufficient collector and mapper counts. For each configuration, we report means over 20 independent runs with randomized LOS node selection from cities with population exceeding 1 million.

We select 1/5 of AOI nodes as mappers and a non-overlapping 1/5 as collectors. All mappers and collectors

TABLE II
SIMULATION PARAMETERS

Symbol	Value	Description
i	87°	Orbital inclination
h	530 km	Altitude
B	10 GHz	ISL channel bandwidth
P	5 W	Transmit power
G_t, G_r	62.5 dBi	Transmit/receive antenna gain
N_T	300 K	Noise temperature
λ	1550 nm	Wavelength
V	10 GB	Data volume per collect task
F_R	5	Reduce compression factor
F_M	1	Map compression factor
m_p, r_p	1	Map/reduce processing time factors
t_h	3	Processing overhead per hop

are within the AOI, and we ensure only ascending (or only descending) satellites are selected per job.

B. Distance-Optimized Routing

Standard +Grid routing selects the shortest Manhattan path by hop count, ignoring physical distance. Since inter-plane link distances vary by up to 40% over an orbital period (Equation 2), we propose a *distance-minimizing, hop-preserving* routing algorithm.

1) Algorithm: Each routing decision uses only local information: a node knows its four neighbors, constellation parameters (M, N, i, h), and current time. The algorithm proceeds:

1) Compute shortest horizontal (cross-plane) and vertical (within-plane) directions to destination.
2) Compute inter-plane distances in forward, backward, and current positions.
3) If at polar crossover (both directions larger than current), route horizontally to exit.
4) If forward inter-plane distance is smaller than current, route horizontally.
5) Otherwise, route vertically to defer cross-plane hops until links shorten.

2) Results: Figures 3 and 4 show that distance optimization reduces average path length by 8–10% at 53° inclination and up to 21% at 87° inclination, without increasing hop count. Higher inclination amplifies the benefit because inter-plane distance variation is more pronounced near poles.

C. Map Task Allocation

We compare three allocation strategies:

- **Random**: Each task assigned to a uniformly random available mapper.
- **Eager**: Tasks processed sequentially; each assigned to lowest-cost available mapper.
- **Bipartite**: Optimal assignment via Hungarian algorithm on full cost matrix.

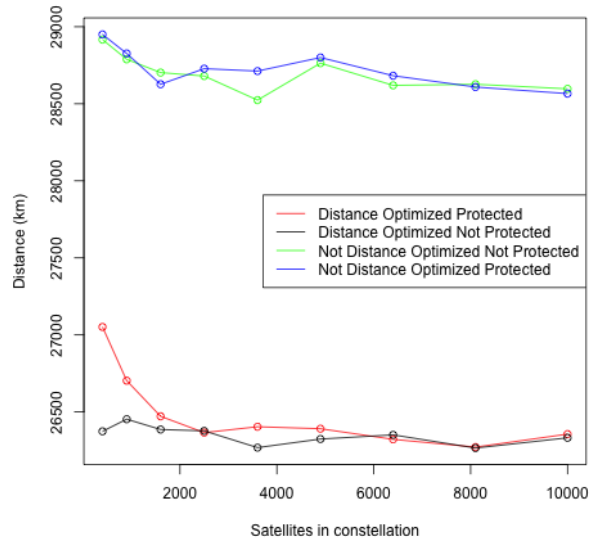

Fig. 3. Routing distance vs. constellation size. Distance optimization reduces path length by 8–10% at 53° inclination, up to 21% at 87°.

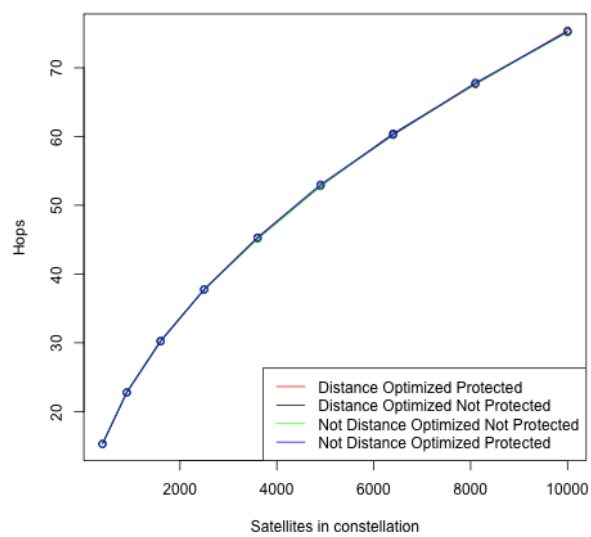

Fig. 4. Hop count comparison. Distance optimization preserves hop count while reducing physical distance.

1) Results: Figure 5 shows that bipartite matching achieves **61–79% improvement** over random allocation and **18–28% improvement** over eager allocation. The eager strategy degrades as task count increases because early greedy choices preclude globally optimal assignments.

D. End-to-End Cost

We evaluate complete SpaceCoMP jobs combining all optimizations.

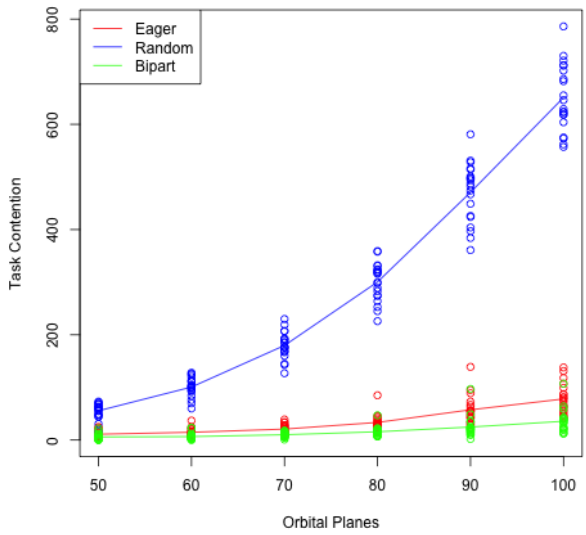

Fig. 5. Map allocation improvement. Bipartite matching achieves 61–79% improvement over random, 18–28% over eager.

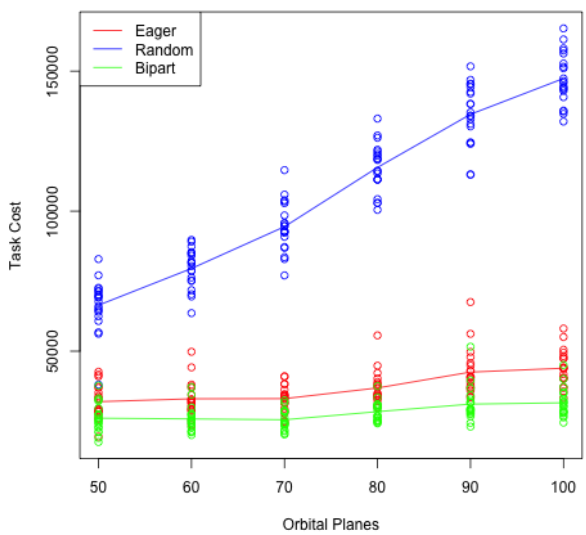

Fig. 6. End-to-end map cost. Bipartite allocation consistently outperforms eager and random baselines.

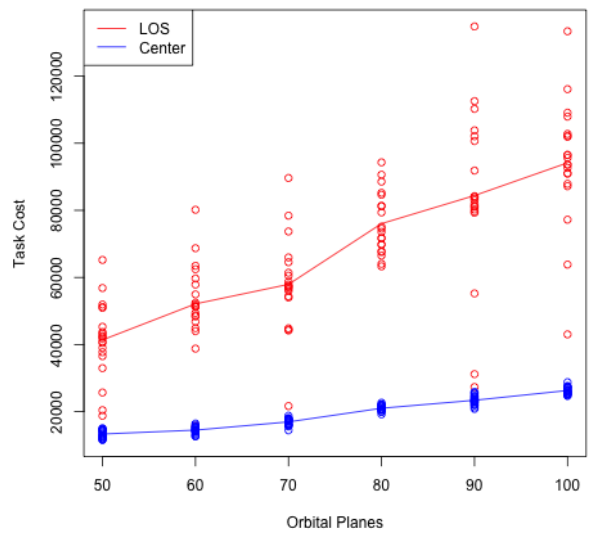

Fig. 7. End-to-end reduce cost. Center-of-AOI placement achieves 67–72% lower cost than LOS placement.

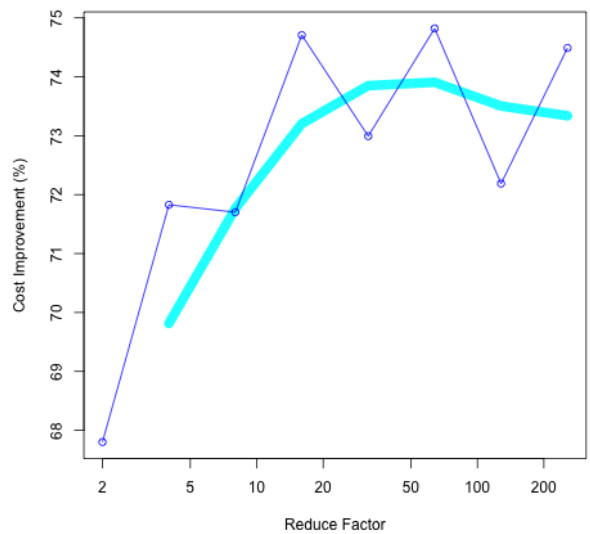

Fig. 8. Reduce cost improvement vs. reduction factor. Benefits increase with compression, saturating around $F_R = 50$.

1) Map Phase: Figure 6 shows end-to-end map cost as orbital plane count increases. Bipartite allocation consistently outperforms baselines across all constellation sizes.

2) Reduce Phase: Figure 7 compares LOS and center-of-AOI reduce placement. Center placement reduces cost by **67–72%** at reduction factor $F_R = 5$.

3) Impact of Reduction Factor: Figure 8 shows how benefits vary with reduction factor F_R. At $F_R = 1$, center placement offers modest benefit; as F_R increases, benefits grow and saturate when mapper-to-reducer transfers dominate.

4) Contention Analysis: Figures 9 and 10 show node visit counts as a proxy for congestion. Bipartite allocation and center placement both reduce contention by distributing traffic more evenly.

E. Summary of Findings

SpaceCoMP's optimizations yield compounding benefits:

- **Routing**: 8–21% distance reduction with zero hop overhead

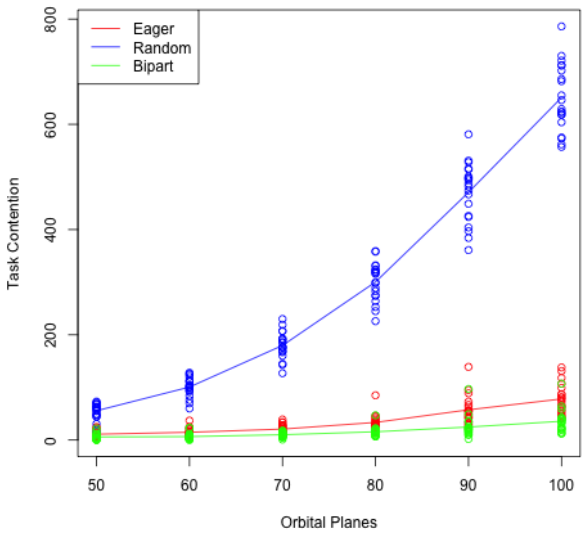

Fig. 9. Map phase contention. Bipartite allocation distributes load more evenly.

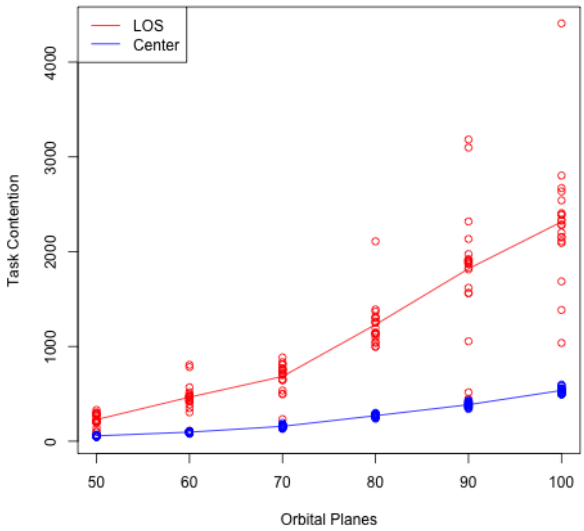

Fig. 10. Reduce phase contention. Center placement reduces hot-spot formation.

- **Map allocation**: 61–79% over random, 18–28% over greedy
- **Reduce placement**: 67–72% cost reduction vs. LOS placement

V. Discussion and Conclusion

In this paper we presented a method similar to tools like MapReduce, in order to simplify the development and deployment of distributed space computing computations. We did so by offering services that take advantage of the unique properties of LEO ISL networks, such as orbital dynamics. Thus, applications do not have to re-implement these features.

We also discussed the architecture and baseline protocol, as well as ran simulations to validate the model. We also contributed a novel routing protocol, a map task allocation scheduler, and a reduce placement strategy.

We expect this SpaceCoMP model to provide value for data-intensive real-time analysis workloads, such as computer vision detection and classification tasks that can be triggered by ground MCP clients via LLM prompts. Those prompts include satellite data and intelligence in model context for dynamic inference combined with other agent data. Hence, the speed at which results can be delivered is of paramount importance, and horizontally scaling over a large number of nodes is an approach that has worked well in data-centers on the ground.

Another interesting use case for satellite earth observation is multi-image super resolution (MISR), whereby many satellites take lower resolution images that are then combined (reduced) into a single higher-resolution image. Clearly, doing the reduction work in orbit would save bandwidth capacity on the downlink. Furthermore, using the approach of this model allows developers to focus on writing sensor collection code, map processing, and reduce aggregation code without the need to re-implement distributed systems infrastructure such as failover, scaling and scheduling.

We also note that recent developments to upgrade satellite hardware to run AI workloads more efficiently, such as neuromorphic computing and vector processing, will help develop more sophisticated map (and reduce) tasks [2]. This part of the recent trend to use space networks and distributed systems running through them in order to disaggregate and simplify space flight system components. This in turn leads to more cost-effective launches and operations, as withness by the improvements achieved in recent years [4].

Recent advancements in mixed consistency [1] could also be utilized in this work to blend globally converging computations and transactional guarantees.

Furthermore, another extension of this work includes a vast improvement in both the security and precision with which the clocks in the constellation can be synchronized. Classical protocols remain vulnerable to spoofing and delay attacks, as timing information can be copied or manipulated in transit. Indeed, successive versions of PTP have required security patches as new attacks are discovered.

Quantum time synchronization offers a promising solution to the security of time synchronization because of the non-cloning nature of quantum states and their inherent resistance to eavesdropping. In addition, quantum mechanisms allow for timing precisions of the order of picoseconds, vastly improving the computing efficiency of the satellite network. Equally important, these quantum mechanisms can be deployed without requiring a master clock outside the LEO constellation.

[2] https://www.nasa.gov/game-changing-development-projects/high-performance-spaceflight-computing-hpsc/

Future work will include implementing the proposed model on the NASA cFS platform[3] using the ColonyOS meta operating system [11]. In our simulations we assumed static costs for running tasks on satellites, whereas real deployments there would be multiple competing task schedulers using the same nodes in order to run multiple tasks. As a consequence we envision embedding more dynamic information in the Task Processor Cost Adjacency Matrix exhibited above such as current load and processing queue size. We could also more explicitly embed cost of contention for each allocation to avoid bottleneck links.

References

[1] J Arns, H Ng, K Psarakis, A Katsifodimos, and P Carbone. Event horizon: Asymmetric dependencies for fast geo-distributed operations. In *CIDR*, 2026.

[2] Vaibhav Bhosale, Ada Gavrilovska, and Ketan Bhardwaj. Krios: Scheduling abstractions and mechanisms for enabling a leo compute cloud. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*, pages 322–340, 2024.

[3] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *Communications of the ACM*, 59(5):50–57, 2016.

[4] Gary Crum, Matthew Dosberg, Evana Gizzi, Cheryl Gramling, Christopher Green, Joanne E Hill, Michael Johnson, Kendall Mauldin, Robert Morgenstern, Christopher Roberts, et al. Nasa's goddard space flight center's distributed systems missions architecture. In *International Astronautical Congress 2022*, 2022.

[5] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

[6] European Union Agency for the Space Programme. IRIS^2: EU secure satellite communication programme. https://www.euspa.europa.eu/eu-space-programme/iris2, 2024.

[7] Mark Handley. Delay is not an option: Low latency routing in space. In *Proceedings of the 17th ACM Workshop on Hot Topics in Networks*, pages 85–91, 2018.

[8] Chalermek Intanagonwiwat, Ramesh Govindan, Deborah Estrin, John Heidemann, and Fabio Silva. Directed diffusion for wireless sensor networking. *IEEE/ACM transactions on networking*, 11(1):2–16, 2003.

[9] Liz Izhikevich, Manda Tran, Katherine Izhikevich, Gautam Akiwate, and Zakir Durumeric. Democratizing LEO satellite network measurement. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 8(1):1–26, 2024.

[10] James Patton Jones and Bill Nitzberg. Scheduling for parallel supercomputing: A historical perspective of achievable utilization. In *Workshop on job scheduling strategies for parallel processing*, pages 1–16. Springer, 1999.

[11] Johan Kristiansson. Colonyos–a meta-operating system for distributed computing across heterogeneous platform. *arXiv preprint arXiv:2403.16486*, 2024.

[12] Jonathan McDowell. Starlink launch statistics. Jonathan's Space Pages, 2024. Accessed: 2024-12-01.

[13] Azalia Mirhoseini, Hieu Pham, Quoc V Le, Benoit Steiner, Rasmus Larsen, Yuefeng Zhou, Naveen Kumar, Mohammad Norouzi, Samy Bengio, and Jeff Dean. Device placement optimization with reinforcement learning. In *International conference on machine learning*, pages 2430–2439. PMLR, 2017.

[14] John K Ousterhout et al. Scheduling techniques for concurrent systems. In *ICDCS*, volume 82, pages 22–30, 1982.

[15] Josep Perdigues et al. HYDRON: The ESA initiative towards optical networking in space. In *European Conference on Optical Communication (ECOC)*, pages 1–4, 2021.

[16] Tobias Pfandzelter and David Bermbach. Celestial: Virtual software system testbeds for the leo edge. In *Proceedings of the 23rd ACM/IFIP International Middleware Conference*, pages 69–81, 2022.

[3]https://etd.gsfc.nasa.gov/capabilities/core-flight-system/

[17] Tobias Pfandzelter and David Bermbach. QoS-aware resource placement for LEO satellite edge computing. In *2022 IEEE 6th International Conference on Fog and Edge Computing (ICFEC)*, pages 66–72. IEEE, 2022.

[18] Tobias Pfandzelter and David Bermbach. Edge computing in low-earth orbit – what could possibly go wrong? In *Proceedings of the 1st ACM Workshop on LEO Networking and Communication*, pages 19–24, 2023.

[19] Fabricio S Prol et al. Position, navigation, and timing (PNT) through low earth orbit (LEO) satellites: A survey. *IEEE Access*, 10:83971–84002, 2022.

[20] Thomas Sandholm, Sayandev Mukherjee, Lin Cheng, and Bernardo A Huberman. Skymemory: A leo edge cache for transformer inference optimization and scale out. *arXiv preprint arXiv:2505.14427*, 2025.

[21] Daniel Selva and David Krejci. A survey and assessment of the capabilities of cubesats for earth observation. *Acta Astronautica*, 74:50–68, 2012.

[22] Reid G Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on computers*, 29(12):1104–1113, 1980.

[23] David Vallado, Paul Crawford, Richard Hujsak, and TS Kelso. Revisiting spacetrack report# 3. *AIAA/AAS Astrodynamics Specialist Conference and Exhibit*, page 6753, 2006.

[24] Johannes Van Der Horst. *Market-based task allocation in distributed satellite systems*. PhD thesis, University of Southampton, 2012.

[25] John G Walker. Satellite constellations. *Journal of the British Interplanetary Society*, 37:559–572, 1984.

[26] Zengshan Yin et al. A comprehensive survey of orbital edge computing: Systems, applications, and algorithms. *Chinese Journal of Aeronautics*, 38(7):103316, 2025.

[27] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Khaled Elmeleegy, Scott Shenker, and Ion Stoica. Delay scheduling: a simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European conference on Computer systems*, pages 265–278, 2010.